Lighthouse Storage Integration for mindX

A complete, self-contained technical reference for integrating **Lighthouse Storage** (perpetual decentralized storage on Filecoin + IPFS) into the mindX autonomous AI agent system. Audience: Python-first agent developers in the PYTHAI/DELTAVERSE ecosystem who also occasionally need JS/TypeScript or raw HTTP code paths.

Table of contents

1. What Lighthouse is and why mindX uses it
2. Architecture, perpetual storage model, and pricing
3. Authentication and API keys
4. SDK ecosystem and installation
5. Uploads — files, folders, buffers, text, encrypted, batch
6. Kavach encryption and access control
7. IPNS — mutable pointers for agent state
8. Filecoin deals, PoDSI, and PDP hot storage
9. Retrieval, gateways, range requests, decryption-on-read
10. File management — list, info, pin, delete
11. Balance, data usage, and pay-per-use (x402)
12. REST endpoint reference
13. Errors, rate limits, retries
14. mindX integration patterns
15. Known limitations and Python/JS parity gaps
16. Resources

---

1. What Lighthouse is and why mindX uses it

Lighthouse Storage is a permanent, decentralized storage protocol powered by Filecoin and IPFS. A one-time payment funds files forever via an on-chain endowment pool that auto-renews Filecoin storage deals. It exposes a developer-friendly REST API plus first-class SDKs for JavaScript/TypeScript, Python, and Go, plus an encryption layer (Kavach) for trust-minimized confidential storage and on-chain access control. Lighthouse Storage +2 + 2

Why mindX cares. Autonomous agents need durable, verifiable, cross-process storage. Lighthouse covers four agent needs in one stack:

- **Persistent agent memory** — write-once content-addressed snapshots survive process restarts and migrate freely between nodes.
- **Encrypted secret storage** — Kavach threshold encryption splits keys across 5 nodes (3-of-5 reconstruction) so no single party (including Lighthouse) can read agent secrets. GitHub lighthouse
- **Mutable identity records via IPNS** — a stable IPNS pointer lets an agent publish "latest state" without minting a new address every update. Lighthouse Lighthouse
- **Token-gated artifact publishing** — outputs can be released only to wallets holding a specific NFT, ERC20 balance, or matching custom contract logic, on EVM and Solana chains alike.

Compared to other web3 storage providers, Lighthouse uniquely combines (a) Filecoin-backed perpetual storage with on-chain endowment renewal, (b) integrated threshold encryption, (c) integrated token-gating across 25+ chains, (d) first-class IPNS, and (e) PoDSI/PDP cryptographic proofs of storage. **Pinata** focuses on IPFS pinning with subscription pricing and no Filecoin proofs. **Web3.Storage / NFT.Storage** dropped their free tiers and pivoted to paid pinning. **Arweave** offers permanent storage but uses its own AR-token economy and lacks native ERC token-gating. **Storj** uses an erasure-coded private network rather than Filecoin's verifiable deal marketplace.

---

2. Architecture, perpetual storage model, and pricing

Lighthouse layers three pieces:

1. **IPFS** for content addressing and hot retrieval through dedicated gateways optimized for 4K video and image-resize. Lighthouse + 2
2. **Filecoin** for cold, cryptographically verifiable storage in sealed sectors held by storage providers (SPs).
3. **Smart-contract endowment pool** that holds user payments and continuously pays SPs to renew deals before expiry.

The endowment mechanic. When a user pays for a file, part of the payment goes immediately to a Filecoin storage deal. The remainder accrues to the **endowment pool** at `0x0E8b07CefDC0363cA6e0Ca06093c2596746f7d3d`. Yield from this pool funds future deal renewals in perpetuity. RaaS (Renewal/Repair/Replication) contracts: **mainnet**

`0xd928b92E6028463910b2005d118C2edE16C38a2a`, calibration `0x4015c3E5453d38Df71539C0F7440603C69784d7a`. PoDSI proofs can be verified on-chain via the V2 Onchain CID Contract `0x5e507e4f223364176D0294D1696226f2405f4EeD`. Lighthouse

Two storage tiers. PoRep (Proof of Replication) is the classic Filecoin sealed-sector tier — best for cold archival of multi-GB datasets. **PDP** (Proof of Data Possession), live since October 2025, targets hot retrieval (dApp frontends, AI inference assets, agent caches) with a 100 MB per-deal limit and ~20-minute deal availability. mindX agents typically want PDP for working memory and PoRep for long-term archives.

File-size constraints. Single-request upload max is **24 GB**. PDP deals max **100 MB**. Replication is configurable: `num_copies` defaults to 2 and may go up to 3. Lighthouse + 2

Pricing (as of April 2026). Three monthly tiers at `lighthouse.storage/pricing`: **Free Trial 5 GB at $0**, **Lite 500 GB at $12/month**, **Premium 2.5 TB at $49/month**. Annual and lifetime tiers exist; encryption, token-gating, IPNS, and migration are sold as add-ons on lower tiers and bundled in Premium. Beyond plans, Lighthouse offers two pay-as-you-go modes: **Filecoin First** (deposit to the endowment pool address, USD credit applied per deal) and **x402** (per-upload USDC micro-payment on Base). Lighthouse Storage + 4

---

3. Authentication and API keys

Lighthouse uses two distinct credentials:

- **API key** — a Bearer token used for all upload, list, deal, and IPNS endpoints. Created by signing a wallet message.
- **Signed message or JWT** — a per-wallet, per-session credential used for **Kavach** encryption flows (uploadEncrypted, fetchEncryptionKey, shareFile, applyAccessCondition). The JWT path is recommended when a single agent makes many encrypted requests. Lighthouse

Getting an API key — UI flow

Sign in at **https://files.lighthouse.storage/**, open the **API Key** tab, click **Create**, label it (e.g., `mindx-prod-agent-01`), and copy the key — **the key is shown only once.** Lighthouse +2 Lighthouse

Getting an API key — programmatic flow

Two-step:

```
GET  https://api.lighthouse.storage/api/auth/get_message?publicKey=<wallet>
POST https://api.lighthouse.storage/api/auth/create_api_key
     body: { publicKey, signedMessage, keyName }
```

Lighthouse

Python (web3.py / eth_account):

```python
import os, requests
from eth_account import Account
from eth_account.messages import encode_defunct

WALLET_PUB  = os.environ["MINDX_WALLET_ADDRESS"]
WALLET_PRIV = os.environ["MINDX_WALLET_PRIVATE_KEY"]

# 1. fetch verification message
msg = requests.get(
    "https://api.lighthouse.storage/api/auth/get_message",
    params={"publicKey": WALLET_PUB},
    timeout=15,
).json()  # plain string

# 2. sign locally
acct = Account.from_key(WALLET_PRIV)
signed = acct.sign_message(encode_defunct(text=msg)).signature.hex()

# 3. exchange for API key
resp = requests.post(
    "https://api.lighthouse.storage/api/auth/create_api_key",
    json={"publicKey": WALLET_PUB, "signedMessage": signed, "keyName": "mindx-agent-
    timeout=15,
).json()
api_key = resp["data"]["apiKey"]
print(api_key)
```

JavaScript / TypeScript:

```js
```

```
import axios from 'axios';
import { ethers } from 'ethers';
import lighthouse from '@lighthouse-web3/sdk';

const wallet = new ethers.Wallet(process.env.MINDX_WALLET_PRIVATE_KEY);
const msg = (await axios.get(
  `https://api.lighthouse.storage/api/auth/get_message?publicKey=${wallet.address}`
)).data;
const signed = await wallet.signMessage(msg);
const { data } = await lighthouse.getApiKey(wallet.address, signed);
console.log(data.apiKey);
```

Lighthouse

curl:

```bash
MSG=$(curl -s "https://api.lighthouse.storage/api/auth/get_message?publicKey=$PUB")
SIG=$(node -e "const {Wallet}=require('ethers');new Wallet(process.env.PRIV).signMes
curl -X POST https://api.lighthouse.storage/api/auth/create_api_key \
  -H 'Content-Type: application/json' \
  -d "{\"publicKey\":\"$PUB\",\"signedMessage\":\"$SIG\",\"keyName\":\"mindx-agent-
```

Kavach JWT flow (recommended for encrypted agents)

```js
import { ethers } from "ethers";
import kavach from "@lighthouse-web3/kavach";

const signer        = new ethers.Wallet(process.env.PRIV);
const authMessage   = await kavach.getAuthMessage(signer.address);
const signed        = await signer.signMessage(authMessage.message);
const { JWT, refreshToken, error } = await kavach.getJWT(signer.address, signed);
// pass JWT as the `signedMessage` argument to subsequent SDK calls.
```

The same flow in Python uses `eth_account` to sign and a plain HTTP POST to `https://encryption.lighthouse.storage/api/message/jwt`. **The third auth flavor is Passkey/WebAuthn** — register at `POST /api/passkey/register/{challenge,verify}`, authenticate at `POST /api/passkey/auth/{challenge,verify}` to receive a Bearer token.

Useful for browser-side agent UIs; not typical for headless mindX deployments. Lighthouse GitHub

Best practice for mindX: store the API key in an OS keyring or `.env` loaded via `pydantic-settings`, never hardcode. Treat the wallet private key as a high-value secret — encrypt it at rest and ideally hold it in an HSM or a dedicated agent-signer service. Lighthouse Lighthouse

---

4. SDK ecosystem and installation

SDK	Package	Latest	License	Repo
Python (priority for mindX)	`lighthouseweb3`	**0.1.5** (Jun 19, 2025)	GPL-3.0	github.com/lighthouse-web3/lighthouse-python-sdk
JavaScript/TypeScript	`@lighthouse-web3/sdk`	**0.4.x** (Mar 25, 2026 tag 0.4.5) github	MIT	github.com/lighthouse-web3/lighthouse-package
Kavach (encryption)	`@lighthouse-web3/kavach`	**0.2.1** (Jan 25, 2025) github	AGPL-3.0 github	github.com/lighthouse-web3/encryption-sdk
Go SDK + lhctl CLI	`lighthouse-go-sdk`	Jan 26, 2026 GitHub github	MIT	github.com/lighthouse-web3/lighthouse-go-sdk
CLI (ships with JS SDK global install)	binary `lighthouse-web3`	bundled	MIT	same as JS SDK

Install commands:

```bash
```

```
# Python
pip install lighthouseweb3

# JS/TS
npm install @lighthouse-web3/sdk
npm install @lighthouse-web3/kavach   # only if you need low-level encryption helper

# CLI (global)
npm install -g @lighthouse-web3/sdk          # binary: lighthouse-web3
# Go CLI alternative
git clone https://github.com/lighthouse-web3/lighthouse-go-sdk && cd lighthouse-go-s
go build -o lhctl ./cmd/lhctl
```

Python initialization pattern (use this in mindX):

```python
import os
from lighthouseweb3 import Lighthouse

# Reads LIGHTHOUSE_TOKEN env var if no arg supplied
lh = Lighthouse(token=os.environ.get("LIGHTHOUSE_TOKEN"))
```

JavaScript:

```js
import lighthouse from '@lighthouse-web3/sdk';
const apiKey = process.env.LIGHTHOUSE_API_KEY;
```

MCP tooling. Lighthouse ships an **MCP server** (`lighthouse-agent-tooling` repo, October 2025) so AI agents and IDE assistants can call upload/fetch/encrypt operations through the Model Context Protocol. mindX agents that already speak MCP can wire this in without touching the SDK directly.

---

5. Uploads — files, folders, buffers, text, encrypted, batch

All uploads ultimately POST multipart form data to `https://upload.lighthouse.storage/api/v0/add` with header `Authorization: Bearer <API_KEY>`. The standard response shape is: Lighthouse

```json
{ "data": { "Name": "<filename>", "Hash": "<CID>", "Size": "<bytes-as-string>" } }
```

Lighthouse

`Hash` is the IPFS CID. Maximum single-request size: **24 GB**. For directories the response carries the **root directory CID**; sub-files resolve as `https://gateway.lighthouse.storage/ipfs/<root>/<filename>`. Lighthouse Lighthouse

5.1 Single file

Python:

```python
import os
from lighthouseweb3 import Lighthouse

lh = Lighthouse(token=os.environ["LIGHTHOUSE_TOKEN"])
try:
    resp = lh.upload(source="/path/to/agent_memory.json")
    cid = resp["data"]["Hash"]
    print(f"CID={cid}  size={resp['data']['Size']}")
    # Optional tag for retrieval grouping
    lh.upload(source="/path/to/agent_memory.json", tag="mindx-memory-v1")
except Exception as e:
    print(f"Upload failed: {e}")
```

Lighthouse

JavaScript:

```js
import lighthouse from '@lighthouse-web3/sdk';

const resp = await lighthouse.upload(
  '/path/to/agent_memory.json',
  process.env.LIGHTHOUSE_API_KEY
);
console.log(resp.data.Hash);
```

Lighthouse

curl:

```bash
curl -X POST -H "Authorization: Bearer $LIGHTHOUSE_API_KEY" \
  -F "file=@/path/to/agent_memory.json" \
  https://upload.lighthouse.storage/api/v0/add
```

Lighthouse

5.2 Directory upload

The SDKs auto-detect directories. Pass a directory path to `lh.upload()` or `lighthouse.upload()` and you receive the root CID. Lighthouse

```python
resp = lh.upload(source="/var/mindx/snapshots/2026-04-28")
print("Root CID:", resp["data"]["Hash"])
```

```js
const resp = await lighthouse.upload('/var/mindx/snapshots/2026-04-28', apiKey);
```

```bash
# multiple -F file=@... parts produce a wrap-with-directory CID
curl -X POST -H "Authorization: Bearer $LIGHTHOUSE_API_KEY" \
  -F "file=@./snap/a.json" -F "file=@./snap/b.json" -F "file=@./snap/c.json" \
  https://upload.lighthouse.storage/api/v0/add
```

5.3 Buffer / text / JSON

The Python SDK uses `uploadBlob(file_like, name, tag="")` for in-memory data. JS exposes `uploadBuffer` and `uploadText` separately. Lighthouse

Python — upload a JSON snapshot from memory:

```python
```

```
import io, json
from lighthouseweb3 import Lighthouse

lh = Lighthouse(token=os.environ["LIGHTHOUSE_TOKEN"])
snapshot = {"agent_id": "mindx-7", "ts": 1745875200, "memory": [...]}
buf = io.BytesIO(json.dumps(snapshot).encode("utf-8"))
resp = lh.uploadBlob(buf, "snapshot.json", tag="mindx-snapshot")
print(resp["data"]["Hash"])
```

JavaScript — text and buffer:

```js
// Text/JSON
const out1 = await lighthouse.uploadText(
  JSON.stringify(snapshot), apiKey, "snapshot.json"
);
// Binary buffer
import { readFileSync } from 'fs';
const out2 = await lighthouse.uploadBuffer(readFileSync('./model.bin'), apiKey);
```

5.4 Multi-file batch

Browser JS supports a `multi=true` flag; Node and Python pass a directory path. curl repeats `-F file=@...`. All produce a single root CID. Lighthouse

5.5 Upload progress (JS only)

```js
const progressCb = (p) => {
  // verbatim formula from docs (note: inverted from intuitive percent)
  const pct = 100 - (p.total / p.uploaded).toFixed(2);
  console.log(`progress=${pct}`);
};
const out = await lighthouse.upload(file, apiKey, null, progressCb);
```

Lighthouse

The Python SDK does not expose a progress callback — wrap your file-like object in a `tqdm`-style reader if you need progress. Lighthouse

5.6 CAR file uploads (recommended for large datasets)

```bash
npx ipfs-car pack ./mindx-corpus --output corpus.car
curl -X POST -H "Authorization: Bearer $LIGHTHOUSE_API_KEY" \
  -F "file=@corpus.car" \
  https://upload.lighthouse.storage/api/v0/add
```

JS SDK v0.4.4+ exposes a native CAR upload helper with progress tracking. No equivalent in the Python SDK as of v0.1.5 — fall back to the curl path or the SDK's `upload()` of a `.car` file (it streams the bytes correctly).

5.7 Configurable Filecoin deal parameters

Pass a `dealParameters` object to `lighthouse.upload(path, apiKey, false, dealParams)`:

```js
const dealParams = {
  num_copies: 2,            // 1..3
  repair_threshold: 28800,  // epochs
  renew_threshold: 240,     // epochs
  miner: ["t017840"],       // preferred SPs
  network: 'calibration',   // 'mainnet' | 'calibration'
};
await lighthouse.upload('/path/file.bin', apiKey, false, dealParams);
```

Lighthouse

Python lacks a typed param object; pass the same dict via raw HTTP if needed.

5.8 Encrypted uploads (preview — full coverage in §6)

```js
// JS — full-featured
const signed = await signAuthMessage(pub, priv);
const out = await lighthouse.uploadEncrypted(path, apiKey, pub, signed);
```

Lighthouse

```python
```

```
# Python — surface is narrower; if uploadEncrypted is unavailable in your version,
# encrypt locally with AES-256-GCM and upload the ciphertext via lh.upload(),
# storing the key separately in your vault. See §15.
```

---

6. Kavach encryption and access control

Kavach is Lighthouse's threshold-encryption SDK (`@lighthouse-web3/kavach`). A 32-byte master key is split into **5 shards** by Shamir-style threshold cryptography; **any 3 of 5** reconstruct the key. Shards are stored across five distributed encryption nodes at `https://encryption.lighthouse.storage/api/.../{1..5}`. No single node holds the master key. Each node independently evaluates the access-control policy you attach before releasing its shard, making the entire system non-custodial. GitHub

6.1 Encrypted upload — JavaScript (full-fidelity path)

```js
import lighthouse from '@lighthouse-web3/sdk';
import { ethers } from 'ethers';

const signAuthMessage = async (pub, priv) => {
  const signer = new ethers.Wallet(priv);
  const msg = (await lighthouse.getAuthMessage(pub)).data.message;
  return signer.signMessage(msg);
};

const pub  = process.env.MINDX_WALLET_ADDRESS;
const priv = process.env.MINDX_WALLET_PRIVATE_KEY;
const sig  = await signAuthMessage(pub, priv);

const resp = await lighthouse.uploadEncrypted(
  '/path/to/secret.json', process.env.LIGHTHOUSE_API_KEY, pub, sig
);
console.log('Encrypted CID:', resp.data[0].Hash);
```

Lighthouse

For text payloads use `lighthouse.textUploadEncrypted(text, apiKey, pub, sig, name?)`. Lighthouse

6.2 Encrypted upload — Python

The Python SDK exposes `uploadEncrypted` in recent builds but its surface is much narrower than JS. The portable pattern is:

```python
# Pattern A — if lh.uploadEncrypted is available
signed = sign_auth_message(WALLET_PUB, WALLET_PRIV)  # see §3
resp = lh.uploadEncrypted(
    source="/path/secret.json",
    publicKey=WALLET_PUB,
    signedMessage=signed,
)
cid = resp["data"][0]["Hash"]
```

```python
# Pattern B — portable fallback. Encrypt locally, upload ciphertext, persist
# the AES key in mindX's secret vault. Use this when Python SDK encryption
# helpers are unavailable or you want full control.
import os, json, secrets
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

key   = AESGCM.generate_key(bit_length=256)
nonce = secrets.token_bytes(12)
plain = json.dumps({"openai_api_key": "sk-..."}).encode()
ct    = AESGCM(key).encrypt(nonce, plain, None)

with open("/tmp/blob.bin", "wb") as f:
    f.write(nonce + ct)
resp = lh.upload("/tmp/blob.bin")
cid  = resp["data"]["Hash"]
mindx_vault.put(f"lh:{cid}", key.hex())   # your own KV
```

6.3 Decryption (full reconstruction)

JavaScript:

```js
```

```
const sig    = await signAuthMessage(pub, priv);
const keyObj = await lighthouse.fetchEncryptionKey(cid, pub, sig);
const blob   = await lighthouse.decryptFile(cid, keyObj.data.key, "application/json"
```

Python:

```python
key_obj = lh.fetchEncryptionKey(cid, WALLET_PUB, signed)
master  = key_obj["data"]["key"]
plain_bytes = lh.decryptFile(cid, master)
open("out.json","wb").write(plain_bytes)
```

Low-level Kavach (when you want manual shard handling):

```js
import { recoverShards, recoverKey } from '@lighthouse-web3/kavach';
const { shards } = await recoverShards(addr, cid, jwt, 3);
const { masterKey } = await recoverKey(shards);
```

github

6.4 Sharing and revoking

```js
// Share with one or more wallets
await lighthouse.shareFile(ownerPub, ["0xRecipient1", "0xRecipient2"], cid, sig);

// Revoke
await lighthouse.revokeFileAccess(ownerPub, ["0xRecipient1"], cid, sig);

// Inspect current ACL
const cond = await lighthouse.getAccessConditions(cid);
```

Python mirrors these as `lh.shareFile(...)`, `lh.revokeFileAccess(...)`, `lh.getAccessConditions(cid)` when those helpers are present in your installed version. Otherwise hit `https://encryption.lighthouse.storage/api/encryption/{share,revoke,access_condition}` per node directly.

6.5 Token-gated and conditional access

Conditions are an array of objects evaluated server-side by each Kavach node before releasing its shard. The schema:

```ts
{
  id: number,                    // unique within array; referenced by aggregator
  chain: string,                 // e.g. "Ethereum","Polygon","FVM","Sepolia"
  method: string,                // "balanceOf" | "ownerOf" | "getBalance" | "getBlockN
  standardContractType: "" | "ERC20" | "ERC721" | "ERC1155" | "Custom",
  contractAddress?: string,
  parameters?: any[],            // ":userAddress" is replaced by requester's address
  returnValueTest: { comparator: "==" | "!=" | ">" | ">=" | "<" | "<=", value: string },
  // Custom only:
  inputArrayType?: string[],     // e.g. ["bytes32","address"]
  outputType?: string            // e.g. "uint256","bool"
}
```

Aggregator string uses `[id]` references with `and`/`or` and parentheses: `"([1] and [2]) or [3]"`. Single-condition shorthand: `"([1])"`.

Examples — gate by NFT ownership:

```js
const conditions = [{
  id: 1, chain: "Ethereum",
  method: "balanceOf", standardContractType: "ERC721",
  contractAddress: "0xMINDX_NFT",
  parameters: [":userAddress"],
  returnValueTest: { comparator: ">", value: "0" },
}];
await lighthouse.applyAccessCondition(pub, cid, sig, conditions, "([1])");
```

ERC20 minimum balance: same shape with `standardContractType: "ERC20"`. Native chain balance: `method: "getBalance"`, `standardContractType: ""`. Time gate: `method: "getBlock"` with `returnValueTest.value` set to a unix timestamp. Custom view function: `standardContractType: "Custom"` plus `inputArrayType`/`outputType`. Astar

Python:

```python
```

```
conditions = [{
    "id": 1, "chain": "Polygon",
    "method": "balanceOf", "standardContractType": "ERC721",
    "contractAddress": "0xMINDX_NFT",
    "parameters": [":userAddress"],
    "returnValueTest": {"comparator": ">", "value": "0"},
}]
lh.applyAccessCondition(WALLET_PUB, cid, signed, conditions, "([1])")
```

Supported chains for conditions: Ethereum, Sepolia, Goerli, Rinkeby, Polygon, Mumbai, Optimism, OptimismGoerli, OptimismKovan, Arbitrum_Sepolia, BSC, BSCTest, AVAX, Fuji, Fantom, FantomTest, FVM, Wallaby, Calibration, Hyperspace, Shardeum, BTTC, BTTC_Testnet, Sepolia_PGN, BASE_Goerli. Solana DEVNET/TESTNET/MAINNET via `chainType: "Solana"`. Coreum and Radix have separate condition shapes documented in the Kavach docs. Lighthouse

> ⚠️ **Owner-zero footgun.** Calling `accessControl(...)` with `decryptionType: "ACCESS_CONDITIONS"` *transfers ownership to address(0)*. After that, only condition-passers can decrypt — even the original owner cannot bypass the conditions. Use this deliberately and never on agent secrets you may need recovery access to.

6.6 Kavach low-level methods reference

Method	Signature	Purpose
`generate(threshold=3, keyCount=5)`	→ `{masterKey, keyShards}` github	Random shard generation GitHub
`shardKey(key, threshold=3, keyCount=5)`	→ `{isShardable, keyShards}` github	Shard a known key
`recoverKey(keyShards)`	→ `{masterKey, error}` github	Reconstruct master
`saveShards(addr, cid, auth, shards, shareTo?)`	→ `{isSuccess, error}`	Persist to 5 nodes
`recoverShards(addr, cid, auth, keyCount=3, dynamicData?)`	→ `{shards, error}` github	Pull shards back GitHub
`shareToAddress(addr, cid, auth, shareTo[])`	→ `{isSuccess, error}`	Direct share

Method	Signature	Purpose
`revokeAccess(addr, cid, auth, revokeTo[])`	→ `{isSuccess, error}`	Direct revoke
`accessControl(addr, cid, auth, conditions, aggregator?, chainType?, keyShards?, decryptionType?)`	→ `{isSuccess, error}`	Apply gating
`getAuthMessage(addr)`	→ `{message, error}` github	Auth nonce
`getJWT(addr, signed, useAsRefreshToken=false)`	→ `{JWT, refreshToken, error}` github	Issue JWT
`transferOwnership(addr, cid, newOwner, auth, resetSharedTo=true)`	→ `{result, error}` github	Transfer file owner

7. IPNS — mutable pointers for agent state

IPNS lets a stable name (the hash of a public key) point to a rotating CID. Perfect for agent identity, "current state" pointers, model checkpoints, and feed-style outputs. Lighthouse

Four operations map to four SDK calls, all backed by `https://api.lighthouse.storage/api/ipns/...`:

SDK	Generate	Publish/Update	List
JS	`lighthouse.generateKey(apiKey)`	`lighthouse.publishRecord(cid, ipnsName, apiKey)`	`lighthouse.`
HTTP	`GET /api/ipns/generate_key`	`GET /api/ipns/publish_record?cid=&keyName=`	`GET /api/ipns/g`

`generateKey` returns `{ipnsName, ipnsId}`. **`ipnsName` is the hash you must pass back to `publishRecord` as `keyName`.** `ipnsId` is the public k51-style identifier callers use to resolve the pointer. Lighthouse

Python (full lifecycle, REST-direct since the Python SDK's IPNS surface is partial):

```python
import os, requests

API = "https://api.lighthouse.storage/api/ipns"
HDR = {"Authorization": f"Bearer {os.environ['LIGHTHOUSE_TOKEN']}"}

# 1. Generate a stable key once per agent
gk = requests.get(f"{API}/generate_key", headers=HDR).json()
ipns_name = gk["data"]["ipnsName"]    # store in your agent state
ipns_id   = gk["data"]["ipnsId"]      # share this; it's the public pointer

# 2. Publish (initial state)
def publish(cid: str):
    return requests.get(
        f"{API}/publish_record",
        headers=HDR,
        params={"cid": cid, "keyName": ipns_name},
    ).json()

publish("Qmd5MBBScDUV3Ly8qahXtZFqyRRfYSmUwEcxpYcV4hzKfW")

# 3. Update — same call, new CID
publish("QmanCeGkwsaCUHaNT24ndriYTYSwZuAy4JDifdYZpHdmRa")

# 4. List & remove
print(requests.get(f"{API}/get_ipns_records", headers=HDR).json())
requests.delete(f"{API}/remove_key", headers=HDR, params={"keyName": ipns_name})

# Resolve from anywhere:
# https://gateway.lighthouse.storage/ipns/<ipns_id>
```

JavaScript:

```js
const k = await lighthouse.generateKey(apiKey);
await lighthouse.publishRecord(cid, k.data.ipnsName, apiKey);
const all = await lighthouse.getAllKeys(apiKey);
await lighthouse.removeKey(k.data.ipnsName, apiKey);
```

curl:

```bash
curl -H "Authorization: Bearer $K" "https://api.lighthouse.storage/api/ipns/generate
curl -H "Authorization: Bearer $K" "https://api.lighthouse.storage/api/ipns/publish_
curl -H "Authorization: Bearer $K" "https://api.lighthouse.storage/api/ipns/get_ipns
curl -X DELETE -H "Authorization: Bearer $K" "https://api.lighthouse.storage/api/ipn
```

mindX use cases: agent identity card (`/ipns/<id>` returns the latest manifest), rotating "memory pointer" updated each tick, reproducible model-checkpoint feed, audit trail of agent decisions where each new CID is itself stored on Lighthouse.

---

8. Filecoin deals, PoDSI, and PDP hot storage

A successful upload returns a CID immediately, but the underlying Filecoin deal can take **hours to a full day** to seal. mindX agents should never block on deal availability; instead poll asynchronously. Lighthouse

8.1 Deal status

JS: `lighthouse.dealStatus(cid)`. Python: `lh.getDealStatus(cid)`. HTTP: `GET https://api.lighthouse.storage/api/lighthouse/deal_status?cid=<cid>`.

```python
deals = lh.getDealStatus("bafy...")
# Each entry: {DealID, Provider, dealStatus, pieceCID, pieceSize, startEpoch, endEpo
sealed = [d for d in deals.get("data", []) if d.get("dealStatus","").startswith("Sea
```

```js
const deals = await lighthouse.dealStatus("bafy...");
```

```bash
curl "https://api.lighthouse.storage/api/lighthouse/deal_status?cid=$CID"
```

8.2 PoDSI — Proof of Data Segment Inclusion

PoDSI is a Merkle proof that your payload is included in the aggregated piece committed to a Filecoin SP. Available within a few minutes of upload — use it for verifiable receipts before deals fully seal.

```python
import requests
proof = requests.get(
    "https://api.lighthouse.storage/api/lighthouse/get_proof",
    params={"cid": cid, "network": "mainnet"},  # or "testnet" for Calibration
).json()
deal_info = proof["dealInfo"]                   # list of {dealId, storageProvider,
```

```js
const proof = (await axios.get(
  "https://api.lighthouse.storage/api/lighthouse/get_proof",
  { params: { cid, network: "mainnet" } }
)).data;
```

The Onchain CID Contract at `0x5e507e4f223364176D0294D1696226f2405f4EeD` exposes `store(cid, config)` and `requestStorageStatus(cid)` so a mindX smart-contract path can record proofs on-chain. GitHub

8.3 PDP — hot storage with proof of data possession

PDP (live since October 2025) suits hot retrieval workloads. Submit a deal request and poll status:

```bash
curl -X GET "https://api.lighthouse.storage/api/lighthouse/pdp_deal_request?cid=$CID
  -H "Authorization: Bearer $LIGHTHOUSE_API_KEY"
curl -X GET "https://api.lighthouse.storage/api/lighthouse/pdp_deal_status?cid=$CID"
```

Response includes `dealStatus`, `RootCID`, `DownloadURL`, and `TXHash`. Limit: 100 MB per deal; expect ~20 minutes to availability. Lighthouse Lighthouse

---

9. Retrieval, gateways, range requests, decryption-on-read

Default Lighthouse gateways:

- IPFS: `https://gateway.lighthouse.storage/ipfs/<cid>`
- IPNS: `https://gateway.lighthouse.storage/ipns/<ipnsId>`
- Image resize: append `?h=200&w=800` Lighthouse

> The public `gateway.lighthouse.storage` is **restricted to premium users**. Free-tier users must use the dedicated URL shown in the dashboard profile. Public IPFS gateways (`ipfs.io`, `dweb.link`, `w3s.link`, `<cid>.ipfs.nftstorage.link`) resolve any Lighthouse CID as a fallback.

Python — direct, streaming, range:

```python
import requests
url = f"https://gateway.lighthouse.storage/ipfs/{cid}"

# direct download
open("out.bin","wb").write(requests.get(url, timeout=60).content)

# streaming with range (first 1 MB)
with requests.get(url, headers={"Range": "bytes=0-1048575"}, stream=True) as r:
    with open("part.bin","wb") as f:
        for chunk in r.iter_content(65536):
            f.write(chunk)

# via SDK
content, meta = lh.download(cid)  # returns (bytes, metadata)
```

JavaScript — axios stream / fetch range:

```js
import axios from 'axios';
import fs from 'fs';

const r = await axios({ url: `https://gateway.lighthouse.storage/ipfs/${cid}`, respo
r.data.pipe(fs.createWriteStream('./out.bin'));

// browser/Node 18+ fetch with Range
const part = await fetch(`https://gateway.lighthouse.storage/ipfs/${cid}`, {
  headers: { Range: 'bytes=0-65535' }
});
```

curl:

```bash
```

```bash
curl -o out.bin "https://gateway.lighthouse.storage/ipfs/$CID"
curl -H "Range: bytes=0-1023" "https://gateway.lighthouse.storage/ipfs/$CID" -o head
```

Decrypt-on-retrieval combines fetch-key and decrypt-file from §6.3. The same flow runs in browser, Node, and Python — assemble shards (3 of 5), recover the master key, decrypt the bytes.

---

10. File management — list, info, pin, delete

List uploads (paginated):

```python
print(lh.getUploads())                        # first page
print(lh.getUploads("YOUR_CID_TO_CHECK"))     # filter by CID
```

```js
const page1 = await lighthouse.getUploads(apiKey);
const page2 = await lighthouse.getUploads(apiKey, page1.data.lastKey);
```

HTTP: `GET https://api.lighthouse.storage/api/user/files_uploaded?lastKey=<cursor>`.

File info:

```js
const info = await lighthouse.getFileInfo(cid);
// { fileName, fileSizeInBytes, encryption, mimeType, dealIDs, ... }
```

```bash
curl "https://api.lighthouse.storage/api/lighthouse/file_info?cid=$CID"
```

Pin existing CID — `lighthouse.pinCid(cid, apiKey)` (JS), CLI `lighthouse-web3 pin`, or POST to the corresponding API route. Migration tooling supports importing **up to 10,000 CIDs in a single request** with auto-verification.

Delete a file — CLI `lighthouse-web3 delete-file <fileID>` or `lhctl --delete <id>`. Note: Lighthouse storage is **perpetual** — delete removes the entry from the user's dashboard/index but Filecoin SP-sealed copies persist until their deals expire (and the endowment pool stops

renewing them). Treat delete as "stop publishing this content," not as cryptographic erasure. GitHub + 2

---

11. Balance, data usage, and pay-per-use (x402)

```python
print(lh.getBalance())   # data usage and remaining credit
```

```js
const balance = await lighthouse.userDataUsage(apiKey);
const wallet  = await lighthouse.getBalance(walletAddress);
```

```bash
curl -H "Authorization: Bearer $LIGHTHOUSE_API_KEY" \
  https://api.lighthouse.storage/api/user/data_usage
```

Filecoin First (pay-per-deal). Deposit to endowment pool address `0x0E8b07CefDC0363cA6e0Ca06093c2596746f7d3d`; USD-equivalent credit applies to your wallet for deal creation. Profile/balance: `GET https://filecoin-first.lighthouse.storage/api/v1/user/get_profile` with `Authorization: API_KEY`. Lighthouse Lighthouse

x402 (pay-per-upload USDC on Base): each upload performs an on-chain micro-payment via the `x402-fetch` library against `POST <api>/api/x402/upload`. Useful for ephemeral agents that need storage without holding an API key. Lighthouse

```js
import { wrapFetchWithPayment } from 'x402-fetch';
import { createWalletClient, http } from 'viem';
import { privateKeyToAccount } from 'viem/accounts';
import { baseSepolia } from 'viem/chains';

const account = privateKeyToAccount(process.env.TEST_PRIVATE_KEY);
const client  = createWalletClient({ account, chain: baseSepolia, transport: http()
const fetchWithPay = wrapFetchWithPayment(fetch, client);
const res = await fetchWithPay(`${API_URL}/api/x402/upload`, { method: 'POST', body:
```

12. REST endpoint reference

Hosts: `api.lighthouse.storage` (control plane), `upload.lighthouse.storage` (uploads), `gateway.lighthouse.storage` (retrieval), `encryption.lighthouse.storage` (Kavach), `node.lighthouse.storage` (legacy), `filecoin-first.lighthouse.storage` (pay-per-deal).

Method	Path	Host	Auth	Purp
GET	`/api/auth/get_message`	api	none	Get s mess publ
POST	`/api/auth/create_api_key`	api	signed	Crea
POST	`/api/v0/add`	upload	API key	File u (mul
GET	`/api/lighthouse/file_info`	api	API key	File r cid=
GET	`/api/user/files_uploaded`	api	API key	List u last
POST	`/api/user/file_delete`	api	API key	Dele id=
GET	`/api/user/data_usage`	api	API key	Data
GET	`/api/lighthouse/get_balance`	api	API key	Wall
GET	`/api/lighthouse/deal_status`	api	none	Filec cid=
GET	`/api/lighthouse/get_proof`	api	none	PoDS cid=
GET	`/api/lighthouse/pdp_deal_request`	api	API key	Subr (?c
GET	`/api/lighthouse/pdp_deal_status`	api	none	PDP cid=
GET	`/api/ipns/generate_key`	api	API key	New
GET	`/api/ipns/publish_record`	api	API key	Publi IPNS cid=
GET	`/api/ipns/get_ipns_records`	api	API key	List I

Method	Path	Host	Auth	Purp
DELETE	`/api/ipns/remove_key`	api	API key	Rem (`?ke`
GET	`/ipfs/{cid}`	gateway	none (premium)	IPFS `h=&w`
GET	`/ipns/{ipnsId}`	gateway	none	IPNS
GET	`/api/message/{address}`	encryption	none	Kava mess
POST	`/api/message/jwt`	encryption	signed	Issue
POST	`/api/setSharedKey/{1..5}`	encryption	signed/JWT	Save N
POST	`/api/retrieveSharedKey/{1..5}`	encryption	signed/JWT	Pull s node
POST	`/api/encryption/share`	encryption	signed/JWT	Shar CID
POST	`/api/encryption/revoke`	encryption	signed/JWT	Revo
POST	`/api/encryption/access_condition`	encryption	signed/JWT	Appl cond
POST	`/api/passkey/register/{challenge,verify}`	api	varies	Web regis
POST	`/api/passkey/auth/{challenge,verify}`	api	varies	Web Bear
GET	`/api/v1/user/get_profile`	filecoin-first	`Authorization: API_KEY`	FF ba
POST	`/api/x402/upload`	x402	USDC payment	Pay-p uplo

13. Errors, rate limits, retries

Lighthouse does not publish a formal error-code or rate-limit table. Observed conventions:

- **401** — missing/invalid API key.
- **403** — gateway tier restriction (free user hitting `gateway.lighthouse.storage`).
- **404** — CID/file not found, or expired record.
- **413** — payload too large (single-request cap is 24 GB; PDP deals 100 MB).
- **429** — implied rate limit; back off and retry.
- **5xx** — transient; retry with exponential backoff.

Documented timing windows:

- Filecoin deals: a few hours up to ~1 day to seal. Treat as eventually consistent.
- PoDSI: a few minutes after upload. Lighthouse
- PDP: ~20 minutes to availability.

Retry policy for mindX agents: wrap every Lighthouse call in tenacity-style retry with jittered exponential backoff (3–5 attempts). Tag transient classes (`requests.Timeout`, `5xx`, `429`) as retryable; treat `4xx` (other than 408/425/429) as permanent. For Kavach, accept partial success: reconstruction needs only 3 of 5 shards, so at most 2 node calls may fail before the operation is considered failed.

Support channels: mail@lighthouse.storage, Discord (`https://discord.com/invite/lighthouse`), Twitter/X `@LighthouseWeb3`, "Talk to Expert" form on lighthouse.storage. lighthouse lighthouse

---

14. mindX integration patterns

These patterns are written against the Python SDK as the primary surface. They assume `lh = Lighthouse(token=os.environ["LIGHTHOUSE_TOKEN"])` and helpers `sign_auth_message()`, `mindx_vault` (your secrets KV), `mindx_kv` (your local agent KV) are already defined.

14.1 Persistent agent memory snapshots

```python
```

```python
import io, json, time

def snapshot_memory(agent_id: str, memory: dict) -> str:
    """Write an agent memory snapshot, return its CID."""
    buf = io.BytesIO(json.dumps({
        "agent_id": agent_id,
        "ts": int(time.time()),
        "schema": "mindx.memory.v1",
        "memory": memory,
    }).encode())
    resp = lh.uploadBlob(buf, f"{agent_id}-mem-{int(time.time())}.json",
                         tag=f"mindx-mem-{agent_id}")
    return resp["data"]["Hash"]

def restore_memory(cid: str) -> dict:
    content, _meta = lh.download(cid)
    return json.loads(content)
```

14.2 Encrypted secret vault

Use the local-encrypt-then-upload pattern (Pattern B in §6.2) when the Python SDK's encryption helpers are unavailable. Store the AES key alongside the CID so the agent — and only the agent — can decrypt later.

```python
def store_secret(name: str, secret: dict) -> str:
    key   = AESGCM.generate_key(bit_length=256)
    nonce = secrets.token_bytes(12)
    ct    = AESGCM(key).encrypt(nonce, json.dumps(secret).encode(), None)
    buf   = io.BytesIO(nonce + ct)
    cid   = lh.uploadBlob(buf, f"{name}.enc", tag="mindx-secret")["data"]["Hash"]
    mindx_vault.put(f"lh-key:{cid}", key.hex())
    return cid

def load_secret(cid: str) -> dict:
    blob, _ = lh.download(cid)
    nonce, ct = blob[:12], blob[12:]
    key = bytes.fromhex(mindx_vault.get(f"lh-key:{cid}"))
    return json.loads(AESGCM(key).decrypt(nonce, ct, None))
```

For full Kavach (no local key custody, threshold network), use the JS SDK or call the Kavach REST endpoints directly — see §6.

14.3 Public artifact publishing

```python
def publish_artifact(path: str, label: str) -> dict:
    resp = lh.upload(source=path, tag=f"mindx-artifact-{label}")
    cid = resp["data"]["Hash"]
    return {
        "cid": cid,
        "gateway": f"https://gateway.lighthouse.storage/ipfs/{cid}",
        "size": int(resp["data"]["Size"]),
    }
```

14.4 Token-gated agent outputs (NFT-gated insight feed)

Use the JS SDK to apply the gate; agents in Python can drive the JS path via subprocess or via direct HTTP to `encryption.lighthouse.storage/api/encryption/access_condition`.

```js
const conditions = [{
  id: 1, chain: "Polygon",
  method: "balanceOf", standardContractType: "ERC721",
  contractAddress: process.env.MINDX_ACCESS_NFT,
  parameters: [":userAddress"],
  returnValueTest: { comparator: ">", value: "0" },
}];
await lighthouse.applyAccessCondition(pub, cid, sig, conditions, "([1])");
```

Holders of the gating NFT can call `fetchEncryptionKey + decryptFile` from any chain client. Lighthouse

14.5 Mutable agent identity via IPNS

```python
```

```python
class AgentIdentity:
    def __init__(self, ipns_name: str, ipns_id: str):
        self.ipns_name, self.ipns_id = ipns_name, ipns_id

    @classmethod
    def create(cls):
        gk = requests.get("https://api.lighthouse.storage/api/ipns/generate_key",
                          headers=HDR).json()["data"]
        return cls(gk["ipnsName"], gk["ipnsId"])

    def update(self, manifest_cid: str):
        requests.get("https://api.lighthouse.storage/api/ipns/publish_record",
                     headers=HDR,
                     params={"cid": manifest_cid, "keyName": self.ipns_name})

    @property
    def public_url(self) -> str:
        return f"https://gateway.lighthouse.storage/ipns/{self.ipns_id}"
```

Persist `ipns_name` once; rotate `manifest_cid` on every state change. External services and other agents subscribe to `public_url` and always see the latest manifest.

14.6 IPFS-anchored on-chain references

Combine PoDSI with the Onchain CID Contract (`0x5e507e4f...EeD`) to put a verifiable anchor on Filecoin EVM:

```python
# 1. Upload artifact
cid = lh.upload("/path/agent_audit.json")["data"]["Hash"]
# 2. Wait minutes, fetch PoDSI
proof = requests.get(
    "https://api.lighthouse.storage/api/lighthouse/get_proof",
    params={"cid": cid, "network": "mainnet"}).json()
# 3. Submit on-chain via your web3 client (call Onchain-CID-Contract.store(cid, conf
```

Now any consumer can verify on-chain that the agent's output was committed to Filecoin storage at a given block.

14.7 Suggested mindX module layout

```
mindx/
└── storage/
    └── lighthouse/
        ├── __init__.py            # exposes LighthouseAdapter
        ├── adapter.py             # thin async wrapper around lighthouseweb3
        ├── kavach_http.py         # raw HTTP for
encryption/share/revoke/conditions
        ├── ipns.py                # AgentIdentity + IPNS helpers
        ├── proofs.py              # deal_status / podsi / pdp helpers
        ├── secret_vault.py        # local-AES fallback for encrypted storage
        └── retry.py               # tenacity policy + error mapping
```

The adapter exposes these high-level methods to the rest of mindX: `put_blob`, `put_file`, `put_dir`, `get`, `pin`, `list`, `info`, `deals`, `podsi`, `ipns_create`, `ipns_publish`, `secret_put`, `secret_get`, `gate_with_nft`. All async; all retry-wrapped.

---

15. Known limitations and Python/JS parity gaps

The Python SDK (`lighthouseweb3` 0.1.5) is materially behind the JS SDK (`@lighthouse-web3/sdk` 0.4.x). Snyk currently classifies it as "Inactive" between the May 2023 v0.1.1 tag and the June 2025 v0.1.5 release. Confirmed gaps in Python that mindX must work around:

- No first-class **`uploadEncrypted`** / **`textUploadEncrypted`** in older Python builds — fall back to the local-encrypt pattern in §14.2 or the JS SDK.
- No **`shareFile`**, **`revokeFileAccess`**, **`applyAccessCondition`**, **`getAccessConditions`**, **`fetchEncryptionKey`**, **`decryptFile`** in stable Python — use raw HTTP against `encryption.lighthouse.storage/api/...`.
- No **`getProofs`** PoDSI helper — call the REST endpoint directly.
- No **`pinCid`** helper — use REST.
- No **`uploadBuffer`** / **`uploadText`** — use `uploadBlob(BytesIO(...), name)`.
- No **CAR-file uploader** with progress.
- No **upload progress callback**.
- No **wallet management** (createWallet etc. — those live in the JS CLI).

Other limitations across the platform:

- **Single-request upload max 24 GB**; PDP deals max 100 MB. Lighthouse github
- **Public gateway** is now restricted to premium accounts; free tier requires the dashboard-issued dedicated URL. Lighthouse
- **Resumable / chunked uploads** are not exposed — large transfers rely on streamed multipart and CAR pre-packing.
- **Delete is logical, not cryptographic** — sealed Filecoin copies persist until deals expire.
- **No published rate-limit table or formal error-code reference.**
- `accessControl(decryptionType="ACCESS_CONDITIONS")` **zeroes the owner** — irreversible; use deliberately.
- Filecoin deal availability is hours-to-a-day; design async pipelines.

---

16. Resources

Official:

- Docs (current Docusaurus): `https://docs.lighthouse.storage`
- Docs (legacy GitBook, still served): `https://docs.lighthouse.storage/lighthouse-1/...`
- Files dApp / API key management: `https://files.lighthouse.storage`
- Pricing: `https://lighthouse.storage/pricing`
- Status / explorer: linked from the docs sidebar
- Email: `mail@lighthouse.storage`
- Discord: `https://discord.com/invite/lighthouse`
- Twitter/X: `@LighthouseWeb3`

GitHub (`github.com/lighthouse-web3`):

- `lighthouse-package` — JS SDK + CLI GitHub
- `lighthouse-python-sdk` — `lighthouseweb3` PyPI source
- `encryption-sdk` — Kavach GitHub
- `lighthouse-go-sdk` — Go SDK + `lhctl` GitHub
- `lighthouse-agent-tooling` — MCP server for AI agents
- `lighthouse-migration-tooling` — bulk pin import GitHub
- `Onchain-CID-Contract` — FVM CID registry

- `x402` — pay-per-use upload demo
- `gitbook` — public docs source

Smart-contract addresses:

- Endowment pool: `0x0E8b07CefDC0363cA6e0Ca06093c2596746f7d3d`
- RaaS mainnet: `0xd928b92E6028463910b2005d118C2edE16C38a2a`
- RaaS calibration: `0x4015c3E5453d38Df71539C0F7440603C69784d7a`
- Onchain CID Contract V2: `0x5e507e4f223364176D0294D1696226f2405f4EeD`
- Aggregator (calibration): `0x01ccBC72B2f0Ac91B79Ff7D2280d79e25f745960`

Package versions (April 2026 snapshot):

- `lighthouseweb3` 0.1.5 (PyPI, Jun 2025) PyPI
- `@lighthouse-web3/sdk` 0.4.x (npm; GitHub tag 0.4.5 Mar 2026) github
- `@lighthouse-web3/kavach` 0.2.1 (Jan 2025) github
- `@lighthouse-web3/lighthouse-cli` — **does not exist as a separate package**; CLI binary `lighthouse-web3` ships with the JS SDK global install. The legacy npm package `lighthouse-web3` is deprecated; do not use.

This document is intended for direct inclusion in mindX project documentation. A developer reading only this file should have everything required to integrate Lighthouse into a mindX agent for persistent memory, encrypted secret storage, public artifact publishing, token-gated outputs, mutable identity records via IPNS, and on-chain anchored references.